1. React Introduction

React គឺជា library JavaScript ដ៏ខ្លាំងក្លាសម្រាប់ការបង្កើតចំណុចប្រទាក់អ្នកប្រើប្រាស់។ វាត្រូវបានគេប្រើយ៉ាងទូលំទូលាយសម្រាប់ការបង្កើតកម្មវិធីគេហទំព័រដែលមានលក្ខណៈឌីណាមិក, លឿន, និងមានស្ថានភាពអាចពង្រីកបាន។ ផ្នែកនេះផ្តល់នូវទិដ្ឋភាពទូទៅអំពី React, លក្ខណៈពិសេសរបស់វា, និងរបៀបចាប់ផ្តើមប្រើវា។

អត្ថន័យ៖ React គឺជា library មួយសម្រាប់បង្កើតចំណុចប្រទាក់អ្នកប្រើប្រាស់នៅលើគេហទំព័រ។

1.1 What is React?

React គឺជា **JavaScript library** សម្រាប់ការបង្កើតចំណុចប្រទាក់អ្នកប្រើប្រាស់, ជាពិសេសសម្រាប់កម្មវិធីគេហទំព័រដែលមានទំព័រតែមួយ (SPA)។ វាត្រូវបានអភិវឌ្ឍន៍ដោយ **Facebook** (ឥឡូវ Meta) ហើយត្រូវបានប្រើយ៉ាងទូលំទូលាយសម្រាប់ការបង្កើតកម្មវិធីគេហទំព័រដែលមានលក្ខណៈឌីណាមិក, លឿន, និងអាចពង្រីកបាន។ React អនុញ្ញាតឱ្យអ្នកអភិវឌ្ឍន៍បង្កើត component UI ដែលអាចប្រើឡើងវិញ និងធ្វើបច្ចុប្បន្នភាព UI ដោយប្រសិទ្ធភាពនៅពេលទិន្នន័យផ្លាស់ប្តូរ។

អត្ថន័យ៖ React គឺជា library សម្រាប់បង្កើតចំណុចប្រទាក់អ្នកប្រើប្រាស់នៅលើគេហទំព័រ។

ចំណុចសំខាន់៖

- React គឺជា **library**, មិនមែនជា framework ពេញលេញទេ។
- វាប្រើ **component-based architecture**។
- React ធ្វើតាម **declarative approach**, ធ្វើឱ្យវាងាយស្រួលក្នុងការសាកល្បង និងយល់។

1.2 Why Use React?

React មានប្រយោជន៍ជាច្រើនសម្រាប់ការអភិវឌ្ឍន៍គេហទំព័រ។ ខាងក្រោមនេះគឺជាហេតុផលមួយចំនួនដែលអ្នកគួរប្រើ React៖

អត្ថន័យ៖ ហេតុអ្វីប្រើ React?

- **Reusable Components:** React អនុញ្ញាតឱ្យអ្នកបង្កើត component ដែលអាចប្រើឡើងវិញបាន, ដែលធ្វើឱ្យការអភិវឌ្ឍន៍កាន់តែរហ័ស និងងាយស្រួល។
- **Virtual DOM:** React ប្រើ Virtual DOM ដើម្បីធ្វើបច្ចុប្បន្នភាព UI ដោយប្រសិទ្ធភាព, ធ្វើឱ្យកម្មវិធីរបស់អ្នកលឿនជាងមុន។
- **Strong Community:** React មានសហគមន៍ធំដែលគាំទ្រនិងផ្តល់នូវធនធានជាច្រើនសម្រាប់អ្នកអភិវឌ្ឍន៍។

1.3 React Features

React មានលក្ខណៈពិសេសជាច្រើនដែលធ្វើឱ្យវាក្លាយជាឧបករណ៍ដ៏ពេញនិយមសម្រាប់ការអភិវឌ្ឍន៍គេហទំព័រ។ ខាងក្រោមនេះគឺជាលក្ខណៈពិសេសសំខាន់ៗរបស់ React៖

អត្ថន័យ៖ លក្ខណៈពិសេសរបស់ React។

- **JSX:** JSX គឺជាស្ពាំភ្ជាប់នៃ JavaScript និង HTML ដែលធ្វើឱ្យការសរសេរកូដកាន់តែងាយស្រួល។
- **Component-Based:** React ប្រើ component-based architecture ដែលធ្វើឱ្យកូដអាចប្រើឡើងវិញបាន។
- **Unidirectional Data Flow:** React ប្រើ unidirectional data flow ដែលធ្វើឱ្យការគ្រប់គ្រង state កាន់តែងាយស្រួល។

1.4 Setting Up React (Vite & CRA)

ដើម្បីចាប់ផ្តើមប្រើ React, អ្នកត្រូវការរៀបចំបរិស្ថានអភិវឌ្ឍន៍។ មានឧបករណ៍ជាច្រើនដែលអាចជួយអ្នកក្នុងការបង្កើតកម្មវិធី React, រួមមាន **Vite** និង **Create React App (CRA)**។

អត្ថន័យ៖ របៀបរៀបចំបរិស្ថានអភិវឌ្ឍន៍សម្រាប់ React។

ជំហានដើម្បីរៀបចំបរិស្ថាន៖

1. **Install Node.js:** ទាញយកនិងដំឡើង Node.js ពីគេហទំព័រផ្លូវការ។

```
# Check Node.js version
node -v
```

2. **Create a React App with Vite:**

```
npm create vite@latest my-react-app --template react
```

3. **Create a React App with CRA:**

```
npx create-react-app my-react-app
```

4. **Run the Development Server:**

```
cd my-react-app
npm start
```

អត្ថន័យ៖ Vite និង CRA គឺជាឧបករណ៍ដែលជួយអ្នកបង្កើតកម្មវិធី React យ៉ាងរហ័ស។

1.5 Writing First React App

បន្ទាប់ពីរៀបចំបរិស្ថានអភិវឌ្ឍន៍រួច, អ្នកអាចចាប់ផ្តើមសរសេរកម្មវិធី React ដំបូងរបស់អ្នក។ ខាងក្រោមនេះគឺជាឧទាហរណ៍សាមញ្ញនៃកម្មវិធី React ដែលបង្ហាញអត្ថបទ "Hello, React!"។

អត្ថន័យ៖ របៀបសរសេរកម្មវិធី React ដំបូង។

ឧទាហរណ៍កូដ៖

```
// App.js
import React from 'react';

function App() {
  return (
    <div>
      <h1>Hello, React!</h1>
    </div>
  );
}

export default App;
```

ជំហានដើម្បីដំណើរការ៖

1. បើកឯកសារ `App.js` នៅក្នុងថតរង្វង់របស់អ្នក។
2. ជំនួសកូដដែលមានដោយឧទាហរណ៍ខាងលើ។
3. រត់សឺរីវអភិវឌ្ឍន៍៖

   ```
   npm start
   ```

4. បើក browser ហើយចូលទៅ `http://localhost:3000` ដើម្បីមើលលទ្ធផល។

លទ្ធផល៖

```
Hello, React!
```

2. React Basics

React Basics គ្របដណ្តប់អំពីគោលគំនិតជាមូលដ្ឋាននៃ React, រួមមាន JSX, ការ render elements, components, props, state, lifecycle, និងការគ្រប់គ្រង events។ ផ្នែកនេះផ្តល់នូវគ្រឹះដ៏រឹងមាំសម្រាប់ការបង្កើតកម្មវិធី React។

អត្ថន័យ៖ ផ្នែកនេះរៀនអំពីគោលគំនិតជាមូលដ្ឋាននៃ React។

2.1 JSX (JavaScript XML)

JSX គឺជាស្ពាំភ្លេចនៃ JavaScript និង HTML ដែលអនុញ្ញាតឱ្យអ្នកសរសេរកូដ HTML នៅក្នុង JavaScript។ វាធ្វើឱ្យការសរសេរ UI កាន់តែងាយស្រួល និងអាចអានបាន។

អត្ថន័យ៖ JSX គឺជាវិធីសរសេរ HTML នៅក្នុង JavaScript។

ឧទាហរណ៍ JSX៖

```
const element = <h1>Hello, JSX!</h1>;
```

ចំណាំ៖ JSX មិនមែនជា JavaScript សុទ្ធនោះទេ ហេតុនេះវាត្រូវបាន compile ទៅជា JavaScript ដោយ Babel។

2.2 Rendering Elements

Rendering elements គឺជាដំណើរការនៃការបង្ហាញ UI នៅលើអេក្រង់។ នៅក្នុង React, អ្នកអាច render elements ដោយប្រើ `ReactDOM.render()`។

អត្ថន័យ៖ របៀបបង្ហាញ UI នៅលើអេក្រង់។

ឧទាហរណ៍ Rendering៖

```
const element = <h1>Hello, React!</h1>;
ReactDOM.render(element, document.getElementById('root'));
```

ចំណាំ៖ `ReactDOM.render()` ត្រូវបានប្រើដើម្បីបង្ហាញ element នៅក្នុង DOM។

2.3 Components & Props

Components គឺជាផ្នែកដែលអាចប្រើឡើងវិញនៃ UI។ អ្នកអាចបង្កើត components ដោយប្រើ functions ឬ classes។ Props គឺជាទិន្នន័យដែលត្រូវបានបញ្ជូនទៅ components។

អត្ថន័យ៖ Components គឺជាផ្នែកនៃ UI ដែលអាចប្រើឡើងវិញ។

ឧទាហរណ៍ Function Component៖

```
function Welcome(props) {
  return <h1>Hello, {props.name}!</h1>;
}
```

ឧទាហរណ៍ Class Component៖

```
class Welcome extends React.Component {
  render() {
```

```
    return <h1>Hello, {this.props.name}!</h1>;
  }
}
```

2.4 State & Lifecycle

State គឺជាទិន្នន័យផ្ទាល់ខ្លួនរបស់ component ដែលអាចផ្លាស់ប្តូរបាន។ Lifecycle methods គឺជាវិធីសាស្ត្រដែលត្រូវបានហៅនៅពេល component ត្រូវបានបង្កើត, ធ្វើបច្ចុប្បន្នភាព, ឬលុបចោល។

អត្ថន័យ៖ State គឺជាទិន្នន័យផ្ទាល់ខ្លួនរបស់ component, ហើយ lifecycle methods គឺជាវិធីសាស្ត្រដែលត្រូវបានហៅនៅពេល component ផ្លាស់ប្តូរ។

ឧទាហរណ៍ State៖

```
class Clock extends React.Component {
  constructor(props) {
    super(props);
    this.state = { date: new Date() };
  }

  render() {
    return <h1>It is {this.state.date.toLocaleTimeString()}.</h1>;
  }
}
```

2.5 Handling Events

Handling events គឺជាការគ្រប់គ្រង events ដូចជា clicks, input changes, និងផ្សេងៗទៀតនៅក្នុង React។ អ្នកអាចប្រើ event handlers ដើម្បីឆ្លើយតបទៅនឹង events។

អត្ថន័យ៖ របៀបគ្រប់គ្រង events នៅក្នុង React។

ឧទាហរណ៍ Event Handling៖

```
function Button() {
  function handleClick() {
    alert('Button clicked!');
  }
```

```
  return <button onClick={handleClick}>Click Me</button>;
}
```

3. React Components

React Components គឺជាផ្នែកសំខាន់នៃកម្មវិធី React។ ពួកវាអនុញ្ញាតឱ្យអ្នកបែងចែក UI ជាផ្នែកតូចៗដែលអាចប្រើឡើងវិញបាន។ ផ្នែកនេះគ្របដណ្តប់អំពី Functional Components, Class Components, ការបញ្ជូន Props, State នៅក្នុង Components, និង Lifecycle Methods។

អត្ថន័យ៖ React Components គឺជាផ្នែកសំខាន់នៃកម្មវិធី React ដែលអាចប្រើឡើងវិញបាន។

3.1 Functional Components

Functional Components គឺជាមុខងារសាមញ្ញដែលទទួល props ហើយ return JSX។ ពួកវាត្រូវបានប្រើយ៉ាងទូលំទូលាយនៅក្នុង React ដោយសារភាពសាមញ្ញ និងភាពងាយស្រួលនៃការប្រើប្រាស់។

អត្ថន័យ៖ Functional Components គឺជាមុខងារសាមញ្ញសម្រាប់បង្កើត UI។

ឧទាហរណ៍ Functional Component៖

```
function Welcome(props) {
  return <h1>Hello, {props.name}!</h1>;
}
```

ចំណាំ៖ Functional Components ត្រូវបានគេប្រើញឹកញាប់ជាង Class Components នៅក្នុង React សម័យទំនើប។

3.2 Class Components

Class Components គឺជា components ដែលត្រូវបានបង្កើតឡើងដោយប្រើ class នៅក្នុង JavaScript។ ពួកវាអាចមាន state និង lifecycle methods។

អត្ថន័យ៖ Class Components គឺជា components ដែលប្រើ class សម្រាប់ការបង្កើត UI។

ឧទាហរណ៍ Class Component៖

```
class Welcome extends React.Component {
  render() {
```

```
    return <h1>Hello, {this.props.name}!</h1>;
  }
}
```

ចំណាំ៖ Class Components ត្រូវបានប្រើក្នុងករណីដែលត្រូវការ state ឬ lifecycle methods។

3.3 Passing Props

Props គឺជាទិន្នន័យដែលត្រូវបានបញ្ជូនពី parent component ទៅ child component។ ពួកវាជាវិធីសម្រាប់ចែករំលែកទិន្នន័យនៅក្នុង React។

អត្ថន័យ៖ Props គឺជាទិន្នន័យដែលត្រូវបានបញ្ជូនពី component មួយទៅ component មួយទៀត។

ឧទាហរណ៍ Passing Props៖

```
function Welcome(props) {
  return <h1>Hello, {props.name}!</h1>;
}

function App() {
  return <Welcome name="Alice" />;
}
```

3.4 State in Components

State គឺជាទិន្នន័យផ្ទាល់ខ្លួនរបស់ component ដែលអាចផ្លាស់ប្តូរបាន។ វាត្រូវបានប្រើដើម្បីគ្រប់គ្រងទិន្នន័យដែលផ្លាស់ប្តូរនៅក្នុង component។

អត្ថន័យ៖ State គឺជាទិន្នន័យផ្ទាល់ខ្លួនរបស់ component។

ឧទាហរណ៍ State៖

```
class Counter extends React.Component {
  constructor(props) {
    super(props);
    this.state = { count: 0 };
  }

  render() {
    return <h1>Count: {this.state.count}</h1>;
  }
}
```

3.5 Component Lifecycle Methods

Lifecycle Methods គឺជាវិធីសាស្ត្រដែលត្រូវបានហៅនៅពេល component ត្រូវបានបង្កើត, ធ្វើបច្ចុប្បន្នភាព, ឬលុបចោល។ ពួកវាអនុញ្ញាតឱ្យអ្នកគ្រប់គ្រងការធ្វើការនៅពេលជាក់លាក់នៅក្នុងអាយុជីវិតរបស់ component។

អត្ថន័យ៖ Lifecycle Methods គឺជាវិធីសាស្ត្រដែលត្រូវបានហៅនៅពេល component ផ្លាស់ប្តូរ។

ឧទាហរណ៍ Lifecycle Methods៖

```
class Clock extends React.Component {
  componentDidMount() {
    this.timerID = setInterval(() => this.tick(), 1000);
  }

  componentWillUnmount() {
    clearInterval(this.timerID);
  }

  tick() {
    this.setState({ date: new Date() });
  }

  render() {
    return <h1>It is {this.state.date.toLocaleTimeString()}.</h1>;
  }
}
```

4. React Hooks

React Hooks គឺជាមុខងារដែលអនុញ្ញាតឱ្យអ្នកប្រើ state និងលក្ខណៈពិសេសផ្សេងៗរបស់ React នៅក្នុង functional components។ ពួកវាត្រូវបានណែនាំនៅក្នុង React 16.8 ដើម្បីធ្វើឱ្យការសរសេរកូដកាន់តែងាយស្រួល និងអាចអានបាន។

អត្ថន័យ៖ React Hooks គឺជាមុខងារសម្រាប់ប្រើ state និងលក្ខណៈពិសេសផ្សេងៗនៅក្នុង functional components។

4.1 useState Hook

useState គឺជា Hook ដែលអនុញ្ញាតឱ្យអ្នកបន្ថែម state ទៅក្នុង functional components។ វាត្រូវបានប្រើយ៉ាងទូលំទូលាយសម្រាប់ការគ្រប់គ្រងទិន្នន័យដែលផ្លាស់ប្តូរនៅក្នុង component។

អត្ថន័យ៖ useState គឺជា Hook សម្រាប់ប្រើ state នៅក្នុង functional components។

ឧទាហរណ៍ useState៖

```
import React, { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>You clicked {count} times</p>
      <button onClick={() => setCount(count + 1)}>Click me</button>
    </div>
  );
}
```

4.2 useEffect Hook

useEffect គឺជា Hook ដែលអនុញ្ញាតឱ្យអ្នកអនុវត្ត side effects នៅក្នុង functional components។ វាត្រូវបានប្រើសម្រាប់ការធ្វើការងារដូចជា fetching data, ការកំណត់ subscriptions, ឬធ្វើបច្ចុប្បន្នភាព DOM។

អត្ថន័យ៖ useEffect គឺជា Hook សម្រាប់អនុវត្ត side effects នៅក្នុង functional components។

ឧទាហរណ៍ useEffect៖

```
import React, { useState, useEffect } from 'react';

function Timer() {
  const [seconds, setSeconds] = useState(0);

  useEffect(() => {
    const interval = setInterval(() => {
      setSeconds(seconds => seconds + 1);
    }, 1000);

    return () => clearInterval(interval);
  }, []);
```

```
  return <p>Seconds: {seconds}</p>;
}
```

4.3 useContext Hook

useContext គឺជា Hook ដែលអនុញ្ញាតឱ្យអ្នកប្រើ context នៅក្នុង functional components។ វាជាវិធីសាមញ្ញសម្រាប់ចែករំលែកទិន្នន័យរវាង components ដោយមិនចាំបាច់បញ្ជូន props តាមដំណាក់កាល។

អត្ថន័យ៖ useContext គឺជា Hook សម្រាប់ប្រើ context នៅក្នុង functional components។

ឧទាហរណ៍ useContext៖

```
import React, { useContext } from 'react';

const ThemeContext = React.createContext('light');

function ThemedButton() {
  const theme = useContext(ThemeContext);
  return <button style={{ background: theme === 'dark' ? 'black' : 'whit
}
```

4.4 useRef Hook

useRef គឺជា Hook ដែលអនុញ្ញាតឱ្យអ្នកបង្កើត reference ទៅកាន់ DOM elements ឬរក្សាទុកតម្លៃដែលមិនផ្លាស់ប្តូរនៅពេល component ធ្វើបច្ចុប្បន្នភាព។

អត្ថន័យ៖ useRef គឺជា Hook សម្រាប់បង្កើត reference ទៅកាន់ DOM elements ឬរក្សាទុកតម្លៃដែលមិនផ្លាស់ប្តូរ។

ឧទាហរណ៍ useRef៖

```
import React, { useRef } from 'react';

function TextInput() {
  const inputRef = useRef(null);

  const focusInput = () => {
    inputRef.current.focus();
  };
```

```
  return (
    <div>
      <input ref={inputRef} type="text" />
      <button onClick={focusInput}>Focus Input</button>
    </div>
  );
}
```

4.5 Custom Hooks

Custom Hooks គឺជាមុខងារដែលអ្នកបង្កើតឡើងដោយខ្លួនឯង ដើម្បីចែករំលែក logic រវាង components។ ពួកវាជាវិធីដ៏មានឥទ្ធិពលដើម្បីធ្វើឱ្យកូដអាចប្រើឡើងវិញបាន។

អត្ថន័យ៖ Custom Hooks គឺជាមុខងារដែលអ្នកបង្កើតឡើងដោយខ្លួនឯងសម្រាប់ចែករំលែក logic។

ឧទាហរណ៍ Custom Hook៖

```
import { useState, useEffect } from 'react';

function useFetch(url) {
  const [data, setData] = useState(null);

  useEffect(() => {
    fetch(url)
      .then(response => response.json())
      .then(data => setData(data));
  }, [url]);

  return data;
}
```

5. React Forms

React Forms ត្រូវបានប្រើសម្រាប់គ្រប់គ្រងការបញ្ចូលពីអ្នកប្រើប្រាស់នៅក្នុងកម្មវិធី React។ ផ្នែកនេះគ្របដណ្ដប់អំពី Controlled Components, Uncontrolled Components, ការគ្រប់គ្រង Forms, និងការធ្វើ Validation។

អត្ថន័យ៖ React Forms គឺជាវិធីសម្រាប់គ្រប់គ្រងការបញ្ចូលពីអ្នកប្រើប្រាស់។

5.1 Controlled Components

Controlled Components គឺជា components ដែលត្រូវបានគ្រប់គ្រងដោយ state របស់ React។ តម្លៃនៃ input ត្រូវបានគ្រប់គ្រងដោយ state, ហើយការផ្លាស់ប្តូរតម្លៃនៃ input ត្រូវបានគ្រប់គ្រងដោយ event handlers។

អត្ថន័យ៖ Controlled Components គឺជា components ដែលត្រូវបានគ្រប់គ្រងដោយ state របស់ React។

ឧទាហរណ៍ Controlled Component៖

```
function ControlledInput() {
  const [value, setValue] = useState('');

  const handleChange = (event) => {
    setValue(event.target.value);
  };

  return (
    <input type="text" value={value} onChange={handleChange} />
  );
}
```

5.2 Uncontrolled Components

Uncontrolled Components គឺជា components ដែលតម្លៃ input ត្រូវបានគ្រប់គ្រងដោយ DOM ជំនួសឱ្យ state របស់ React។ អ្នកអាចចូលប្រើតម្លៃ input ដោយប្រើ ref។

អត្ថន័យ៖ Uncontrolled Components គឺជា components ដែលតម្លៃ input ត្រូវបានគ្រប់គ្រងដោយ DOM។

ឧទាហរណ៍ Uncontrolled Component៖

```
function UncontrolledInput() {
  const inputRef = useRef(null);

  const handleSubmit = () => {
    alert('Input value: ' + inputRef.current.value);
  };
```

```
  return (
    <div>
      <input type="text" ref={inputRef} />
      <button onClick={handleSubmit}>Submit</button>
    </div>
  );
}
```

5.3 Form Handling

Form Handling គឺជាដំណើរការនៃការគ្រប់គ្រងការបញ្ចូលពីអ្នកប្រើប្រាស់នៅក្នុង form។ នៅក្នុង React, អ្នកអាចគ្រប់គ្រង form ដោយប្រើ Controlled Components ឬ Uncontrolled Components។

អត្ថន័យ៖ Form Handling គឺជាការគ្រប់គ្រងការបញ្ចូលពីអ្នកប្រើប្រាស់នៅក្នុង form។

ឧទាហរណ៍ Form Handling៖

```
function LoginForm() {
  const [username, setUsername] = useState('');
  const [password, setPassword] = useState('');

  const handleSubmit = (event) => {
    event.preventDefault();
    alert(`Username: ${username}, Password: ${password}`);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input
        type="text"
        placeholder="Username"
        value={username}
        onChange={(e) => setUsername(e.target.value)}
      />
      <input
        type="password"
        placeholder="Password"
        value={password}
        onChange={(e) => setPassword(e.target.value)}
      />
      <button type="submit">Login</button>
```

```
    </form>
  );
}
```

5.4 Validation in Forms

Validation គឺជាដំណើរការនៃការពិនិត្យថាតើទិន្នន័យដែលបញ្ចូលដោយអ្នកប្រើប្រាស់ត្រឹមត្រូវឬអត់។ នៅក្នុង React, អ្នកអាចធ្វើ validation ដោយប្រើ state និង conditional rendering។

អត្ថន័យ៖ Validation គឺជាការពិនិត្យទិន្នន័យដែលបញ្ចូលដោយអ្នកប្រើប្រាស់។

ឧទាហរណ៍ Validation៖

```
function ValidationForm() {
  const [email, setEmail] = useState('');
  const [error, setError] = useState('');

  const handleSubmit = (event) => {
    event.preventDefault();
    if (!email.includes('@')) {
      setError('Invalid email address');
    } else {
      setError('');
      alert('Form submitted successfully!');
    }
  };

  return (
    <form onSubmit={handleSubmit}>
      <input
        type="email"
        placeholder="Email"
        value={email}
        onChange={(e) => setEmail(e.target.value)}
      />
      {error && <p style={{ color: 'red' }}>{error}</p>}
      <button type="submit">Submit</button>
    </form>
```

```
  );
}
```

6. React Router (Navigation)

React Router គឺជាបណ្ណាល័យដ៏ខ្លាំងក្លាសម្រាប់គ្រប់គ្រងការរុករកនិង routing នៅក្នុងកម្មវិធី React។ វាអនុញ្ញាតឱ្យអ្នកបង្កើតកម្មវិធី single-page applications (SPAs) ដែលមានទិដ្ឋភាពច្រើននិង routing ឌីណាមិក។ ផ្នែកនេះគ្របដណ្តប់អំពីរបៀបដំឡើង React Router, ការកំណត់ routing មូលដ្ឋាន, ការធ្វើការជាមួយ route parameters, និងការអនុវត្ត private routes។

អត្ថន័យ៖ React Router គឺជាបណ្ណាល័យសម្រាប់គ្រប់គ្រងការរុករកនិង routing នៅក្នុងកម្មវិធី React។

6.1 Installing React Router

ដើម្បីប្រើ React Router នៅក្នុងកម្មវិធី React របស់អ្នក, អ្នកត្រូវតែដំឡើងវាជាមុនសិន។ អ្នកអាចដំឡើង React Router ដោយប្រើ npm ឬ yarn។

អត្ថន័យ៖ របៀបដំឡើង React Router។

ឧទាហរណ៍ដំឡើង React Router៖

```
npm install react-router-dom
```

ឬប្រើ yarn៖

```
yarn add react-router-dom
```

6.2 Basic Routing (Route, Link, Navigate)

React Router ផ្តល់នូវ components ដូចជា **Route**, **Link**, និង **Navigate** សម្រាប់ការកំណត់ routing មូលដ្ឋាននៅក្នុងកម្មវិធី React។

អត្ថន័យ៖ របៀបកំណត់ routing មូលដ្ឋានដោយប្រើ Route, Link, និង Navigate។

ឧទាហរណ៍ Basic Routing៖

```
import { BrowserRouter as Router, Route, Routes, Link } from 'react-rout

function App() {
```

```
  return (
    <Router>
      <nav>
        <Link to="/">Home</Link>
        <Link to="/about">About</Link>
      </nav>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
      </Routes>
    </Router>
  );
}

function Home() {
  return <h1>Home Page</h1>;
}

function About() {
  return <h1>About Page</h1>;
}
```

6.3 Route Parameters (useParams)

React Router អនុញ្ញាតឱ្យអ្នកប្រើ route parameters ដើម្បីបង្កើត dynamic routes។ អ្នកអាចចូលប្រើ route parameters ដោយប្រើ hook **useParams**។

អត្ថន័យ៖ របៀបប្រើ route parameters ដើម្បីបង្កើត dynamic routes។

ឧទាហរណ៍ Route Parameters៖

```
import { BrowserRouter as Router, Route, Routes, useParams } from 'react

function App() {
  return (
    <Router>
      <Routes>
        <Route path="/user/:id" element={<User />} />
      </Routes>
    </Router>
```

```
  );
}

function User() {
  const { id } = useParams();
  return <h1>User ID: {id}</h1>;
}
```

6.4 Private Routes & Redirects

Private routes គឺជា routes ដែលតម្រូវឱ្យអ្នកប្រើប្រាស់ចូលប្រើបានតែនៅពេលពួកគេបានចូលប្រើប្រាស់។ អ្នកអាចបង្កើត private routes ដោយប្រើ **Navigate** និង conditional rendering។

អត្ថន័យ៖ របៀបបង្កើត private routes និង redirects។

ឧទាហរណ៍ Private Routes៖

```
import { BrowserRouter as Router, Route, Routes, Navigate } from 'react-

function App() {
  const isAuthenticated = true; // Replace with actual authentication lo

  return (
    <Router>
      <Routes>
        <Route
          path="/dashboard"
          element={isAuthenticated ? <Dashboard /> : <Navigate to="/logi
        />
        <Route path="/login" element={<Login />} />
      </Routes>
    </Router>
  );
}

function Dashboard() {
  return <h1>Dashboard</h1>;
}

function Login() {
```

```
  return <h1>Login Page</h1>;
}
```

7. React Styling

React ផ្តល់វិធីជាច្រើនដើម្បី style components របស់អ្នក, រួមមាន Inline CSS, CSS Modules, Styled Components, និង frameworks utility-first ដូចជា Tailwind CSS។ ផ្នែកនេះគ្របដណ្តប់អំពីវិធីសាស្ត្រផ្សេងៗក្នុងការ style components នៅក្នុង React។

អត្ថន័យ៖ React ផ្តល់វិធីជាច្រើនដើម្បី style components របស់អ្នក, រួមមាន Inline CSS, CSS Modules, Styled Components, និង frameworks utility-first ដូចជា Tailwind CSS។

7.1 Inline CSS

Inline CSS អនុញ្ញាតឱ្យអ្នកអនុវត្ត styles ដោយផ្ទាល់ទៅលើ JSX elements ដោយប្រើ attribute **style**។ Styles ត្រូវបានសរសេរជា JavaScript objects ជាមួយនឹង property names ជា camelCase។

អត្ថន័យ៖ Inline CSS អនុញ្ញាតឱ្យអ្នកអនុវត្ត styles ដោយផ្ទាល់ទៅលើ JSX elements ដោយប្រើ attribute **style**។

ឧទាហរណ៍ Inline CSS៖

```
function InlineStyleExample() {
  const styles = {
    color: 'blue',
    fontSize: '20px',
    backgroundColor: 'lightgray',
  };

  return <div style={styles}>This is an inline-styled div.</div>;
}
```

7.2 CSS Modules

CSS Modules អនុញ្ញាតឱ្យអ្នកសរសេរ CSS នៅក្នុងឯកសារដាច់ដោយឡែក និងចង្អុល styles ទៅកាន់ components ជាក់លាក់។ វាជួយការពារកុំឱ្យ styles មានការប៉ះទង្គិចគ្នានៅក្នុងកម្មវិធីធំៗ។

អត្ថន័យ៖ CSS Modules អនុញ្ញាតឱ្យអ្នកសរសេរ CSS នៅក្នុងឯកសារដាច់ដោយឡែក និងចង្អុល styles ទៅកាន់ components ជាក់លាក់។

ឧទាហរណ៍ CSS Modules៖

```
import styles from './MyComponent.module.css';

function MyComponent() {
  return <div className={styles.myClass}>This is styled with CSS Modules
}
```

ឯកសារ MyComponent.module.css៖

```
.myClass {
  color: red;
  font-size: 18px;
}
```

7.3 Styled Components

Styled Components គឺជា library មួយដែលអនុញ្ញាតឱ្យអ្នកសរសេរ CSS នៅក្នុង JavaScript។ វាប្រើ tagged template literals ដើម្បីបង្កើត components ដែលមាន styles ភ្ជាប់ជាមួយ។

អត្ថន័យ៖ Styled Components គឺជា library សម្រាប់សរសេរ CSS នៅក្នុង JavaScript។

ឧទាហរណ៍ Styled Components៖

```
import styled from 'styled-components';

const StyledDiv = styled.div`
  color: green;
  font-size: 20px;
  background-color: lightyellow;
`;

function StyledComponentExample() {
  return <StyledDiv>This is a styled component.</StyledDiv>;
}
```

7.4 Tailwind CSS

Tailwind CSS គឺជា utility-first CSS framework ដែលផ្តល់នូវ classes ជាច្រើនសម្រាប់ style components ដោយផ្ទាល់នៅក្នុង JSX។ វាធ្វើឱ្យការសរសេរ CSS កាន់តែងាយស្រួល និងរហ័ស។

អត្ថន័យ៖ Tailwind CSS គឺជា utility-first CSS framework សម្រាប់ style components ដោយផ្ទាល់នៅក្នុង JSX។

ឧទាហរណ៍ Tailwind CSS៖

```
function TailwindExample() {
  return (
    <div className="text-blue-500 text-xl bg-gray-100 p-4">
      This is styled with Tailwind CSS.
    </div>
  );
}
```

8. React API Calls

កម្មវិធី React ជាញឹកញាប់ត្រូវការទាញយកទិន្នន័យពី APIs ដើម្បីបង្ហាញខ្លឹមសារឌីណាមិក។ ផ្នែកនេះគ្របដណ្តប់អំពីរបៀបធ្វើ API calls ដោយប្រើ Fetch API និង Axios, ការគ្រប់គ្រង loading និង error states, និងការអនុវត្ត pagination និង infinite scrolling។

អត្ថន័យ៖ React API Calls គឺជាការទាញយកទិន្នន័យពី APIs ដើម្បីបង្ហាញខ្លឹមសារឌីណាមិកនៅក្នុងកម្មវិធី React។

8.1 Fetching Data with Fetch API

Fetch API គឺជា built-in method នៅក្នុង JavaScript សម្រាប់ធ្វើ HTTP requests។ វាងាយស្រួលប្រើ និងមិនត្រូវការ library បន្ថែម។

អត្ថន័យ៖ Fetch API គឺជា method ដែលមានស្រាប់នៅក្នុង JavaScript សម្រាប់ធ្វើ HTTP requests។

ឧទាហរណ៍ Fetch API៖

```
import React, { useEffect, useState } from 'react';

function FetchDataExample() {
  const [data, setData] = useState(null);
```

```
  useEffect(() => {
    fetch('https://jsonplaceholder.typicode.com/posts')
      .then(response => response.json())
      .then(data => setData(data))
      .catch(error => console.error('Error fetching data:', error));
  }, []);

  return (
    <div>
      {data ? (
        <ul>
          {data.map(item => (
            <li key={item.id}>{item.title}</li>
          ))}
        </ul>
      ) : (
        <p>Loading...</p>
      )}
    </div>
  );
}
```

8.2 Fetching Data with Axios

Axios គឺជា library ប្រព័ន្ធទូទៅសម្រាប់ធ្វើ HTTP requests។ វាផ្តល់នូវ features ជាច្រើនដូចជា automatic JSON parsing, request cancellation, និង error handling។

អត្ថន័យ៖ Axios គឺជា library សម្រាប់ធ្វើ HTTP requests ដែលមាន features ជាច្រើន។

ឧទាហរណ៍ Axios៖

```
import React, { useEffect, useState } from 'react';
import axios from 'axios';

function AxiosExample() {
  const [data, setData] = useState(null);

  useEffect(() => {
    axios.get('https://jsonplaceholder.typicode.com/posts')
      .then(response => setData(response.data))
      .catch(error => console.error('Error fetching data:', error));
```

```
  }, []);

  return (
    <div>
      {data ? (
        <ul>
          {data.map(item => (
            <li key={item.id}>{item.title}</li>
          ))}
        </ul>
      ) : (
        <p>Loading...</p>
      )}
    </div>
  );
}
```

8.3 Handling Loading & Errors

នៅពេលធ្វើ API calls, វាសំខាន់ណាស់ក្នុងការគ្រប់គ្រង loading states និង error states។ នេះជួយឱ្យអ្នកប្រើប្រាស់ដឹងថាទិន្នន័យកំពុងត្រូវបានទាញយក ឬមានបញ្ហាកើតឡើង។

អត្ថន័យ៖ ការគ្រប់គ្រង loading និង error states នៅពេលធ្វើ API calls។

ឧទាហរណ៍ Handling Loading & Errors៖

```
import React, { useEffect, useState } from 'react';
import axios from 'axios';

function LoadingErrorExample() {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    axios.get('https://jsonplaceholder.typicode.com/posts')
      .then(response => {
        setData(response.data);
        setLoading(false);
      })
      .catch(error => {
```

```
        setError(error);
        setLoading(false);
      });
  }, []);

  if (loading) return <p>Loading...</p>;
  if (error) return <p>Error: {error.message}</p>;

  return (
    <ul>
      {data.map(item => (
        <li key={item.id}>{item.title}</li>
      ))}
    </ul>
  );
}
```

8.4 Pagination & Infinite Scrolling

Pagination និង Infinite Scrolling គឺជាវិធីសាស្ត្រពីរយ៉ាងដើម្បីគ្រប់គ្រងការបង្ហាញទិន្នន័យច្រើន។ Pagination បែងចែកទិន្នន័យជាទំព័រផ្សេងៗ, រីង Infinite Scrolling ទាញយកទិន្នន័យថ្មីនៅពេលអ្នកប្រើប្រាស់រំកិលទៅផ្នែកខាងក្រោម។

អត្ថន័យ៖ Pagination និង Infinite Scrolling គឺជាវិធីសាស្ត្រពីរយ៉ាងដើម្បីគ្រប់គ្រងការបង្ហាញទិន្នន័យច្រើន។

ឧទាហរណ៍ Pagination៖

```
import React, { useState, useEffect } from 'react';
import axios from 'axios';

function PaginationExample() {
  const [data, setData] = useState([]);
  const [page, setPage] = useState(1);

  useEffect(() => {
    axios.get(`https://jsonplaceholder.typicode.com/posts?_page=${page}&
      .then(response => setData(response.data))
      .catch(error => console.error('Error fetching data:', error));
  }, [page]);

  return (
```

```
    <div>
      <ul>
        {data.map(item => (
          <li key={item.id}>{item.title}</li>
        ))}
      </ul>
      <button onClick={() => setPage(page + 1)}>Next Page</button>
    </div>
  );
}
```

ຂະຫນາດ Infinite Scrolling:

```
import React, { useState, useEffect } from 'react';
import axios from 'axios';

function InfiniteScrollExample() {
  const [data, setData] = useState([]);
  const [page, setPage] = useState(1);

  useEffect(() => {
    axios.get(`https://jsonplaceholder.typicode.com/posts?_page=${page}&
      .then(response => setData(prevData => [...prevData, ...response.da
      .catch(error => console.error('Error fetching data:', error));
  }, [page]);

  useEffect(() => {
    const handleScroll = () => {
      if (window.innerHeight + document.documentElement.scrollTop === do
        setPage(prevPage => prevPage + 1);
      }
    };

    window.addEventListener('scroll', handleScroll);
    return () => window.removeEventListener('scroll', handleScroll);
  }, []);

  return (
    <ul>
      {data.map(item => (
```

```
        <li key={item.id}>{item.title}</li>
      ))}
    </ul>
  );
}
```

9. React State Management

State management គឺជាផ្នែកសំខាន់នៃការបង្កើតកម្មវិធី React ដែលមានស្ថានភាពអាចពង្រីកបាន និងងាយស្រួលថែទាំ។ ផ្នែកនេះគ្របដណ្តប់អំពីបច្ចេកទេសគ្រប់គ្រង state ផ្សេងៗ, រួមមាន useReducer hook, Context API, Redux Toolkit, និង libraries ទំនើបដូចជា Zustand និង Recoil។

អត្ថន័យ៖ State management គឺជាផ្នែកសំខាន់នៃការបង្កើតកម្មវិធី React ដែលមានស្ថានភាពអាចពង្រីកបាន និងងាយស្រួលថែទាំ។

9.1 useReducer Hook

useReducer គឺជា hook ជំនួសឱ្យ **useState** សម្រាប់គ្រប់គ្រង state logic ស្មុគស្មាញ។ វាមានប្រយោជន៍ជាពិសេសនៅពេល state transitions ពឹងផ្អែកលើ state មុន ឬពាក់ព័ន្ធនឹងតម្លៃរងច្រើន។

អត្ថន័យ៖ useReducer គឺជា hook សម្រាប់គ្រប់គ្រង state logic ស្មុគស្មាញ។

ឧទាហរណ៍ useReducer៖

```
import React, { useReducer } from 'react';

const initialState = { count: 0 };

function reducer(state, action) {
  switch (action.type) {
    case 'increment':
      return { count: state.count + 1 };
    case 'decrement':
      return { count: state.count - 1 };
    default:
      throw new Error();
```

```
  }
}

function Counter() {
  const [state, dispatch] = useReducer(reducer, initialState);

  return (
    <div>
      <p>Count: {state.count}</p>
      <button onClick={() => dispatch({ type: 'increment' })}>Increment<
      <button onClick={() => dispatch({ type: 'decrement' })}>Decrement<
    </div>
  );
}
```

9.2 Context API

Context API គឺជាវិធីសាស្ត្រមួយដើម្បីចែករំលែក state រវាង components ដោយមិនចាំបាច់បញ្ជូន props តាមដំណាក់កាល។ វាមានប្រយោជន៍សម្រាប់ការគ្រប់គ្រង global state។

អត្ថន័យ៖ Context API គឺជាវិធីសាស្ត្រចែករំលែក state រវាង components ដោយមិនចាំបាច់បញ្ជូន props។

ឧទាហរណ៍ Context API៖

```
import React, { createContext, useContext } from 'react';

const ThemeContext = createContext();

function App() {
  return (
    <ThemeContext.Provider value="dark">
      <Toolbar />
    </ThemeContext.Provider>
  );
}

function Toolbar() {
  return <ThemedButton />;
}
```

```
function ThemedButton() {
  const theme = useContext(ThemeContext);
  return <button style={{ background: theme === 'dark' ? 'black' : 'whit
}
```

9.3 Redux Toolkit

Redux Toolkit គឺជា library ដែលសម្រួលការប្រើ Redux ដោយផ្តល់នូវ tools និង abstractions ដើម្បីធ្វើឱ្យការគ្រប់គ្រង state កាន់តែងាយស្រួល។

អត្ថន័យ៖ Redux Toolkit គឺជា library សម្រាប់សម្រួលការប្រើ Redux។

ឧទាហរណ៍ Redux Toolkit៖

```
import { configureStore, createSlice } from '@reduxjs/toolkit';
import { Provider, useSelector, useDispatch } from 'react-redux';

const counterSlice = createSlice({
  name: 'counter',
  initialState: { value: 0 },
  reducers: {
    increment: state => {
      state.value += 1;
    },
    decrement: state => {
      state.value -= 1;
    },
  },
});

const store = configureStore({
  reducer: counterSlice.reducer,
});

function Counter() {
  const count = useSelector(state => state.value);
  const dispatch = useDispatch();

  return (
```

```
    <div>
      <p>Count: {count}</p>
      <button onClick={() => dispatch(counterSlice.actions.increment())}
      <button onClick={() => dispatch(counterSlice.actions.decrement())}
    </div>
  );
}

function App() {
  return (
    <Provider store={store}>
      <Counter />
    </Provider>
  );
}
```

9.4 Zustand & Recoil

Zustand និង **Recoil** គឺជា libraries ទំនើបសម្រាប់គ្រប់គ្រង state នៅក្នុង React។ Zustand ផ្តល់នូវ API សាមញ្ញ, រីង Recoil ផ្តោតលើការគ្រប់គ្រង state ដែលមានលក្ខណៈឌីណាមិក។

អត្ថន័យ៖ Zustand និង Recoil គឺជា libraries ទំនើបសម្រាប់គ្រប់គ្រង state។

ឧទាហរណ៍ Zustand៖

```
import create from 'zustand';

const useStore = create(set => ({
  count: 0,
  increment: () => set(state => ({ count: state.count + 1 })),
  decrement: () => set(state => ({ count: state.count - 1 })),
}));

function Counter() {
  const { count, increment, decrement } = useStore();

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={increment}>Increment</button>
```

```
      <button onClick={decrement}>Decrement</button>
    </div>
  );
}
```

ឧទាហរណ៍ Recoil:

```
import { atom, useRecoilState } from 'recoil';

const countState = atom({
  key: 'countState',
  default: 0,
});

function Counter() {
  const [count, setCount] = useRecoilState(countState);

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>Increment</button>
      <button onClick={() => setCount(count - 1)}>Decrement</button>
    </div>
  );
}
```

10. React Performance Optimization

ការធ្វើឱ្យកម្មវិធី React មានស្ថានភាពលឿន និងឆ្លើយតបយ៉ាងល្អ គឺជាការសំខាន់ណាស់។ ផ្នែកនេះគ្របដណ្ដប់អំពីបច្ចេកទេសដូចជា React.memo(), useCallback, useMemo, និង lazy loading ជាមួយ Suspense ដើម្បីកែលម្អស្ថានភាពកម្មវិធី React របស់អ្នក។

អត្ថន័យ: ការធ្វើឱ្យកម្មវិធី React មានស្ថានភាពលឿន និងឆ្លើយតបយ៉ាងល្អ។

10.1 React.memo()

React.memo() គឺជា higher-order component (HOC) ដែលជួយកាត់បន្ថយការ re-render នៃ components នៅពេល props មិនផ្លាស់ប្តូរ។ វាមានប្រយោជន៍សម្រាប់ការបង្កើត components ដែលមានស្ថានភាពល្អ។

អត្ថន័យ៖ React.memo() គឺជា HOC សម្រាប់កាត់បន្ថយការ re-render នៃ components។

ឧទាហរណ៍ React.memo()៖

```
import React, { memo } from 'react';

const MyComponent = memo(function MyComponent(props) {
  return <div>{props.value}</div>;
});

export default MyComponent;
```

10.2 useCallback Hook

useCallback គឺជា hook ដែលជួយកាត់បន្ថយការ re-create functions នៅពេល components re-render។ វាមានប្រយោជន៍សម្រាប់ការបង្កើត callback functions ដែលមានស្ថានភាពល្អ។

អត្ថន័យ៖ useCallback គឺជា hook សម្រាប់កាត់បន្ថយការ re-create functions។

ឧទាហរណ៍ useCallback៖

```
import React, { useState, useCallback } from 'react';

function MyComponent() {
  const [count, setCount] = useState(0);

  const increment = useCallback(() => {
    setCount(prevCount => prevCount + 1);
  }, []);

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={increment}>Increment</button>
    </div>
  );
}
```

10.3 useMemo Hook

useMemo គឺជា hook ដែលជួយកាត់បន្ថយការ re-compute values នៅពេល components re-render។ វាមានប្រយោជន៍សម្រាប់ការគណនាតម្លៃដែលចំណាយពេលច្រើន។

អត្ថន័យ៖ useMemo គឺជា hook សម្រាប់កាត់បន្ថយការ re-compute values។

ឧទាហរណ៍ useMemo៖

```
import React, { useMemo } from 'react';

function MyComponent({ list }) {
  const sortedList = useMemo(() => {
    return list.sort((a, b) => a - b);
  }, [list]);

  return (
    <ul>
      {sortedList.map(item => (
        <li key={item}>{item}</li>
      ))}
    </ul>
  );
}
```

10.4 Lazy Loading & Suspense

Lazy Loading គឺជាបច្ចេកទេសដើម្បី load components តែនៅពេលចាំបាច់, ដែលជួយកាត់បន្ថយការ load ដំបូងនៃកម្មវិធី។ **Suspense** គឺជា component ដែលអនុញ្ញាតឱ្យអ្នកបង្ហាញ fallback UI នៅពេល components កំពុងត្រូវបាន load។

អត្ថន័យ៖ Lazy Loading គឺជាបច្ចេកទេស load components តែនៅពេលចាំបាច់, ហើយ Suspense គឺជា component សម្រាប់បង្ហាញ fallback UI។

ឧទាហរណ៍ Lazy Loading & Suspense៖

```
import React, { lazy, Suspense } from 'react';

const LazyComponent = lazy(() => import('./LazyComponent'));

function MyComponent() {
  return (
    <Suspense fallback={<div>Loading...</div>}>
```

```
      <LazyComponent />
    </Suspense>
  );
}
```

11. Advanced React Concepts

Advanced React concepts ដូចជា Higher-Order Components (HOC), Render Props, Error Boundaries, និង Portals អនុញ្ញាតឱ្យអ្នកបង្កើតកម្មវិធីដែលមានភាពអាចបត់បែន និងរឹងមាំ។ ផ្នែកនេះរុករកគំនិតទាំងនេះយ៉ាងលម្អិត។

អត្ថន័យ៖ Advanced React concepts ដូចជា Higher-Order Components (HOC), Render Props, Error Boundaries, និង Portals អនុញ្ញាតឱ្យអ្នកបង្កើតកម្មវិធីដែលមានភាពអាចបត់បែន និងរឹងមាំ។

11.1 Higher-Order Components (HOC)

Higher-Order Components (HOC) គឺជា function ដែលទទួល component មួយហើយ return component ថ្មីដែលមានលក្ខណៈបន្ថែម។ វាជាវិធីសាស្ត្រមួយដើម្បីចែករំលែក logic រវាង components។

អត្ថន័យ៖ Higher-Order Components (HOC) គឺជា function សម្រាប់បង្កើត components ថ្មីដែលមានលក្ខណៈបន្ថែម។

ឧទាហរណ៍ HOC៖

```
import React from 'react';

function withLoading(WrappedComponent) {
  return function WithLoadingComponent({ isLoading, ...props }) {
    if (isLoading) {
      return <div>Loading...</div>;
    }
    return <WrappedComponent {...props} />;
  };
}

function MyComponent({ data }) {
  return <div>{data}</div>;
}
```

```
const MyComponentWithLoading = withLoading(MyComponent);
```

11.2 Render Props

Render Props គឺជាវិធីសាស្ត្រមួយដើម្បីចែករំលែក logic រវាង components ដោយប្រើ prop ដែលជា function។ វាអនុញ្ញាតឱ្យ components ប្រើ logic ដូចគ្នាប៉ុន្តែ render UI ផ្សេងគ្នា។

អត្ថន័យ៖ Render Props គឺជាវិធីសាស្ត្រចែករំលែក logic រវាង components។

ឧទាហរណ៍ Render Props៖

```
import React from 'react';

function DataProvider({ render }) {
  const data = 'Hello, Render Props!';
  return render(data);
}

function MyComponent() {
  return (
    <DataProvider render={data => <div>{data}</div>} />
  );
}
```

11.3 Error Boundaries

Error Boundaries គឺជា components ដែលចាប់ errors នៅក្នុង child components ហើយបង្ហាញ fallback UI ជំនួសឱ្យការបរាជ័យទាំងស្រុងនៃកម្មវិធី។

អត្ថន័យ៖ Error Boundaries គឺជា components សម្រាប់ចាប់ errors និងបង្ហាញ fallback UI។

ឧទាហរណ៍ Error Boundaries៖

```
import React from 'react';

class ErrorBoundary extends React.Component {
  constructor(props) {
    super(props);
    this.state = { hasError: false };
  }
```

```
  static getDerivedStateFromError(error) {
    return { hasError: true };
  }

  componentDidCatch(error, errorInfo) {
    console.error('Error:', error, errorInfo);
  }

  render() {
    if (this.state.hasError) {
      return <h1>Something went wrong.</h1>;
    }
    return this.props.children;
  }
}

function MyComponent() {
  return (
    <ErrorBoundary>
      <ComponentThatMayError />
    </ErrorBoundary>
  );
}
```

11.4 Portals

Portals គឺជាវិធីសាស្ត្រមួយដើម្បី render children ទៅក្នុង DOM node ដែលនៅខាងក្រៅ hierarchy របស់ parent component។ វាមានប្រយោជន៍សម្រាប់ការបង្កើត modals, tooltips, និង popovers។

អត្ថន័យ៖ Portals គឺជាវិធីសាស្ត្រសម្រាប់ render children ទៅក្នុង DOM node ដែលនៅខាងក្រៅ hierarchy របស់ parent component។

ឧទាហរណ៍ Portals៖

```
import React from 'react';
import ReactDOM from 'react-dom';

function Modal({ children }) {
  return ReactDOM.createPortal(
    <div className="modal">
      {children}
```

```
    </div>,
    document.getElementById('modal-root')
  );
}

function App() {
  return (
    <div>
      <Modal>
        <h1>This is a modal!</h1>
      </Modal>
    </div>
  );
}
```

12. React and Firebase

Firebase គឺជា platform backend-as-a-service ដ៏ខ្លាំងក្លាដែលផ្តល់ features ដូចជា authentication, database, និង hosting។ ផ្នែកនេះគ្របដណ្តប់អំពីរបៀបបញ្ចូល Firebase ជាមួយ React ដើម្បីបង្កើតកម្មវិធី full-stack។

អត្ថន័យ៖ Firebase គឺជា platform backend-as-a-service ដ៏ខ្លាំងក្លាដែលផ្តល់ features ដូចជា authentication, database, និង hosting។

12.1 Firebase Authentication

Firebase Authentication គឺជា service ដែលអនុញ្ញាតឱ្យអ្នកបង្កើតការចូលប្រើប្រាស់ដោយប្រើ email, Google, Facebook, និង providers ផ្សេងៗទៀត។ វាងាយស្រួលប្រើ និងមានសុវត្ថិភាពខ្ពស់។

អត្ថន័យ៖ Firebase Authentication គឺជា service សម្រាប់ការចូលប្រើប្រាស់ដោយប្រើ email, Google, Facebook, និង providers ផ្សេងៗទៀត។

ឧទាហរណ៍ Firebase Authentication៖

```
import { getAuth, signInWithEmailAndPassword } from 'firebase/auth';

const auth = getAuth();
```

```
function login(email, password) {
  signInWithEmailAndPassword(auth, email, password)
    .then((userCredential) => {
      const user = userCredential.user;
      console.log('Logged in user:', user);
    })
    .catch((error) => {
      console.error('Error logging in:', error);
    });
}
```

12.2 Firestore Database

Firestore Database គឺជា NoSQL database ដែលផ្តល់នូវការទុកដាក់ទិន្នន័យជាពេលជាក់លាក់ (real-time)។ វាងាយស្រួលប្រើ និងមាន scalability ខ្ពស់។

អត្ថន័យ៖ Firestore Database គឺជា NoSQL database សម្រាប់ទុកដាក់ទិន្នន័យជាពេលជាក់លាក់។

ឧទាហរណ៍ Firestore Database៖

```
import { getFirestore, collection, addDoc } from 'firebase/firestore';

const db = getFirestore();

async function addData() {
  try {
    const docRef = await addDoc(collection(db, 'users'), {
      name: 'John Doe',
      email: 'john@example.com',
    });
    console.log('Document written with ID:', docRef.id);
  } catch (error) {
    console.error('Error adding document:', error);
  }
}
```

12.3 Hosting React on Firebase

Firebase Hosting គឺជា service ដែលអនុញ្ញាតឱ្យអ្នក deploy និង host កម្មវិធី React របស់អ្នកយ៉ាងងាយស្រួល។ វាផ្តល់នូវ performance ខ្ពស់ និង SSL ដោយស្វ័យប្រវត្តិ។

អត្ថន័យ៖ Firebase Hosting គឺជា service សម្រាប់ deploy និង host កម្មវិធី React។

ជំហានដើម្បី Host React on Firebase៖

1. Install Firebase CLI៖

```
npm install -g firebase-tools
```

2. Login to Firebase៖

```
firebase login
```

3. Initialize Firebase project៖

```
firebase init
```

4. Build your React app៖

```
npm run build
```

5. Deploy to Firebase៖

```
firebase deploy
```

13. React Testing

Testing គឺជាផ្នែកសំខាន់នៃការបង្កើតកម្មវិធី React ដែលអាចទុកចិត្តបាន និងងាយស្រួលថែទាំ។ ផ្នែកនេះគ្របដណ្តប់អំពី unit testing ជាមួយ Jest, component testing ជាមួយ React Testing Library, និង end-to-end testing ជាមួយ Cypress។

អត្ថន័យ៖ Testing គឺជាផ្នែកសំខាន់នៃការបង្កើតកម្មវិធី React ដែលអាចទុកចិត្តបាន និងងាយស្រួលថែទាំ។

13.1 Testing with Jest

Jest គឺជា testing framework ដ៏ពេញនិយមសម្រាប់ unit testing នៅក្នុង JavaScript និង React។ វាផ្តល់នូវ tools ដើម្បីសរសេរ tests, run tests, និងវាយតម្លៃលទ្ធផល។

អត្ថន័យ៖ Jest គឺជា testing framework សម្រាប់ unit testing។

ឧទាហរណ៍ Testing with Jest៖

```
function sum(a, b) {
  return a + b;
}

test('adds 1 + 2 to equal 3', () => {
  expect(sum(1, 2)).toBe(3);
});
```

13.2 React Testing Library

React Testing Library គឺជា library សម្រាប់ test React components ដោយផ្តោតលើអ្វីដែលអ្នកប្រើប្រាស់ឃើញ និងធ្វើ។ វាជួយឱ្យអ្នកសរសេរ tests ដែលជិតស្និទ្ធនឹងការប្រើប្រាស់ពិត។

អត្ថន័យ៖ React Testing Library គឺជា library សម្រាប់ test React components។

ឧទាហរណ៍ React Testing Library៖

```
import { render, screen } from '@testing-library/react';
import MyComponent from './MyComponent';

test('renders MyComponent', () => {
  render(<MyComponent />);
  const linkElement = screen.getByText(/Hello, World!/i);
  expect(linkElement).toBeInTheDocument();
});
```

13.3 End-to-End Testing with Cypress

Cypress គឺជា tool សម្រាប់ end-to-end testing ដែលអនុញ្ញាតឱ្យអ្នក test កម្មវិធីរបស់អ្នកពីចាប់ផ្តើមដល់ចប់។ វាផ្តល់នូវ interface ដ៏ងាយស្រួល និង features ដូចជា time travel debugging។

អត្ថន័យ៖ Cypress គឺជា tool សម្រាប់ end-to-end testing។

ឧទាហរណ៍ End-to-End Testing with Cypress៖

```
describe('My First Test', () => {
  it('Visits the app', () => {
    cy.visit('/');
    cy.contains('Hello, World!');
```

```
  });
});
```

14. React Deployment

Deploying កម្មវិធី React គឺជាជំហានចុងក្រោយដើម្បីធ្វើឱ្យវាអាចចូលប្រើប្រាស់បានដោយអ្នកប្រើប្រាស់។ ផ្នែកនេះគ្របដណ្តប់អំពីរបៀប deploy កម្មវិធី React ទៅកាន់ platforms ដូចជា Vercel និង Netlify, ព្រមទាំងរបៀប build កម្មវិធីសម្រាប់ production។

អត្ថន័យ៖ Deploying កម្មវិធី React គឺជាជំហានចុងក្រោយដើម្បីធ្វើឱ្យវាអាចចូលប្រើប្រាស់បានដោយអ្នកប្រើប្រាស់។

14.1 Deploying to Vercel

Vercel គឺជា platform ដ៏ពេញនិយមសម្រាប់ deploy កម្មវិធី React។ វាផ្តល់នូវការធ្វើ deployment ដោយស្វ័យប្រវត្តិ, performance optimization, និង scalability។

អត្ថន័យ៖ Vercel គឺជា platform សម្រាប់ deploy កម្មវិធី React។

ជំហានដើម្បី Deploy to Vercel៖

1. Install Vercel CLI៖

   ```
   npm install -g vercel
   ```

2. Login to Vercel៖

   ```
   vercel login
   ```

3. Deploy your React app៖

   ```
   vercel
   ```

14.2 Deploying to Netlify

Netlify គឺជា platform មួយទៀតដែលល្អសម្រាប់ deploy កម្មវិធី React។ វាផ្តល់នូវការធ្វើ deployment ដោយស្វ័យប្រវត្តិ, continuous integration, និង serverless functions។

អត្ថន័យ៖ Netlify គឺជា platform សម្រាប់ deploy កម្មវិធី React។

ជំហានដើម្បី Deploy to Netlify៖

1. Install Netlify CLI៖

```
npm install -g netlify-cli
```

2. Login to Netlify៖

```
netlify login
```

3. Deploy your React app៖

```
netlify deploy --prod
```

14.3 Building for Production

មុនពេល deploy កម្មវិធី React របស់អ្នក, អ្នកត្រូវតែ build វាសម្រាប់ production។ នេះជាជំហានសំខាន់ដើម្បីបង្កើត optimized build ដែលត្រៀមខ្លួនសម្រាប់ deployment។

អត្ថន័យ៖ Building for production គឺជាជំហានសំខាន់មុនពេល deploy កម្មវិធី React។

ជំហានដើម្បី Build for Production៖

```
npm run build
```

បន្ទាប់ពី build, ឯកសារនឹងត្រូវបានរក្សាទុកនៅក្នុងថត `build/`។ អ្នកអាច deploy ថតនេះទៅកាន់ platform ណាមួយដូចជា Vercel ឬ Netlify។

អំពីយើង

CodeKhmerLearning ផ្តល់ឱ្យអ្នកនូវវគ្គសិក្សាពិសេសក្នុងការរៀនកូដភាសាខ្មែរ។ យើងមានក្រុមការងារព្រមមានជំនាញក្នុងការអភិវឌ្ឍកម្មវិធី និងវេបសាយ។

© 2025 CodeKhmerLearning. All Rights Reserved.

តំណភ្ជាប់រហ័ស

ទំនាក់ទំនង

សូមទំនាក់ទំនងមកកាន់យើងក្តី:

Email: thymuoyhak.biu@gmail.com

Phone: +855 888 052 812

© 2025 CodeKhmerLearning. All rights reserved.